The Byte Tax: Decode Speed of Byte-Level Language Models on a Consumer GPU

Fabio-Eric Rempel
Independent researcher, Detmold, Germany
ORCID 0009-0008-8447-6159

October 2026

Abstract

Byte-level language models remove the tokenizer, but at batch size 1 they decode more slowly per generated byte than the token model they come from. We measure this *byte tax* for Bolmo 1B, a byteified OLMo 2 1B, against its source on a consumer GPU (RTX 3060 Ti), through a chain of 67 hypotheses written down before each measurement. In eager decoding Bolmo is 2.36× slower per byte. Most of the gap is launch and host overhead in its per-byte local encoder and decoder: CUDA graphs and a single-sync decode loop make Bolmo 3.8× faster, byte-identically, and leave a gap of 1.5× that follows a memory-bandwidth model. Weight quantization confirms the model and shows that the per-byte local stack is the cheapest part to quantize, saving 4.1× more time per gigabyte than the global transformer. Speculative drafting first widens the tax to 2.37×: a token model verifies several drafted tokens almost for free, and a byte model's drafting rounds end and restart at byte granularity. Aligning drafting rounds to patch boundaries reverses this. It makes Bolmo 2.06× faster than its plain loop in bf16, exact in fp32, and narrows the drafted tax to 1.36×, below the undrafted 1.49×. Optimizations of weight reads favour the token model, and at int8 the tax settles at 1.60×, about 13% above the bandwidth floor for that configuration. A kernel census and microbenchmarks show that the remaining overhead is kernel count, not synchronization, and that a persistent megakernel would gain only 4 to 8%. Overall Bolmo becomes 10.3× faster than eager, and 26 of the 67 hypotheses were refuted.

1 Introduction

The speed gap between byte-level and token-level models at batch size 1 is not fixed: it depends on which optimizations are applied, and most standard optimizations favour the token model.

Byte-level models read raw UTF-8 bytes instead of subword tokens. They avoid tokenizer artifacts, handle character-level tasks better, and treat all languages and scripts alike, and recent scaling studies find they can reach a higher quality ceiling than token models (Marathe et al., 2026; Wang et al., 2026). Their main cost has been efficiency: a text has about four times as many bytes as tokens. Latent-tokenizer architectures (Pagnoni et al., 2025; Hwang et al., 2026) answer this by grouping bytes into patches, so a large global transformer runs once per patch while small local layers run once per byte. Byteification (Minixhofer et al., 2026) converts an existing subword model into such an architecture with under 1% of a pretraining budget, which for the first time gives a byte model and a token model that share the same global transformer and training lineage.

That pairing makes a clean measurement possible. We compare Bolmo 1B with OLMo 2 1B, the model it was byteified from, at batch size 1 on a consumer GPU, the setting of local, single-user inference.

We ask:

1. Where does decode time go in the byte model, compared with its source?
2. How much of the gap is overhead that standard techniques remove?
3. Does the remaining gap follow a memory-bandwidth model, and which part of the byte model carries it?
4. Does speculative drafting close the gap or widen it, and why?
5. How close does the byte model get to the hardware floor?

Every step was a written study with hypotheses fixed before measurement; 26 of 67 were refuted, including several of our own explanations, and all are reported in a companion technical report.

Contributions.

- A decomposition of the byte tax into launch overhead, weight bandwidth and per-kernel cost, validated by intervention rather than profiling alone.
- The finding that the per-byte local stack is the most efficient place to quantize a byte model.
- A mechanism for why speculative drafting favours coarse units: the free unit per round and the restart position after a miss.
- Boundary-gated drafting, which aligns rounds to patches and turns drafting from widening the byte tax (2.37×) into narrowing it (1.36×).
- A bandwidth-floor analysis showing the remaining tax at int8 is within about 13% of what this hardware allows.

2 Background and setup

Bolmo and OLMo 2 share their 16-layer global transformer; the byte model adds a small local stack that runs on every byte.

2.1 The models

Bolmo 1B (`allenai/Bolmo-1B`, revision `452690a8`) has four parts. A local encoder (one mLSTM layer (Beck et al., 2025) with an MLP) contextualizes each byte. A boundary predictor decides where patches end during prefill; during decoding the boundary flag is part of the output, using a fused vocabulary of 256 bytes plus 256 byte-with-boundary atoms. The global model, OLMo 2's 16 transformer layers, runs once per patch. A local decoder (four mLSTM layers) turns the patch representation back into byte predictions. Patches average 3.4 to 3.8 bytes, close to a token. OLMo 2 1B (`allenai/OLMo-2-0425-1B`; Team OLMo et al., 2025) runs its full model once per token, including a 100k-entry output head of about 0.21B parameters, about the size of Bolmo's local stack (about 0.18B).

2.2 Hardware and workload

All headline numbers come from one RTX 3060 Ti (8 GB, Windows, measured copy bandwidth 411 GB/s), batch size 1, greedy decoding and a fixed seed. Timing uses five prompts (English, Python, German, JSON, mixed Unicode) with 256 new bytes; OLMo generates 72 tokens (about 259 bytes) so cache sizes match. Drafting parameters were tuned on a separate set of 25 prompts. A Linux Tesla T4 run in fp32 served as a cross-check of the eager ratio (2.09× against 2.25× on Windows).

2.3 Measurement

A small harness hooks each component with CUDA events and also runs every workload without hooks for true throughput (hooks add 5 to 7%). The metric is milliseconds per generated UTF-8 byte. Quality is measured as bits per byte on the first 200,000 bytes of WikiText-2 test and as accuracy on 1,500 items of CUTE (Edman et al., 2024), a character-manipulation benchmark. Optimized outputs are compared byte for byte with eager output; in fp32 all drafted and graphed variants are identical, while bf16 runs diverge only at near-ties of the top two logits.

2.4 Protocol

Each of the 13 hypothesis-driven studies was a written task with hypotheses and the author's predictions copied verbatim into a findings file before any measurement and never edited afterwards. Every speed cell was run twice; pairs differing by more than 5% were rerun. Most studies ended with a cost-model gate: a variant was built only if its predicted gain cleared a threshold set in advance. Experiments were run by Claude Code under these written specifications.

3 Where the time goes

In eager decoding the byte tax is the per-byte local loop, and most of that loop is overhead rather than computation.

3.1 Eager profile

In eager Hugging Face decoding, Bolmo needs 15.0 ms per byte and OLMo 2 6.36 ms (2.36×). The global transformer costs almost the same per byte in both models (about 6.8 against 6.4 ms), because a 3.4-byte patch compresses nearly as well as a 3.6-byte token. The gap is Bolmo's local encoder and decoder: about 9 ms per byte, against 0.7 ms for OLMo's embedding, norm and head. A global layer costs about 1.4 ms per call in both models, roughly 4.4× what its weight reads require. On a Linux T4 trace the GPU was busy 35.5% of the time during Bolmo generation and 63.8% during OLMo generation, with about 430 and 220 kernels per byte.

3.2 Removing overhead

Table 1: Overhead removal for Bolmo 1B, bf16, default prompts, 256 new bytes.

Variant	ms/byte	Speedup vs eager
Eager	15.01	1.00×
CUDA graph, local loop	8.76	1.71×
CUDA graph, local loop and global model	4.85	3.09×
Single-sync decode loop	3.97	3.78×

Capturing the decode step in CUDA graphs removes per-kernel launch work from the CPU; output stays byte-identical to eager (Table 1). The remaining host loop made nine syncs per byte. A replacement loop with one sync per byte, which reads the generated byte and its boundary flag in one copy and computes subword-suffix ids with an Aho–Corasick automaton (exact against the tokenizer on about 30,000 positions), removes it. Contrary to our prediction, the host loop was only 21% of the graphed time; the $O(n)$ history walk we suspected cost 47 μs per call. With the same loop treatment, OLMo 2 reaches 2.58 ms/byte, so the gap is 1.54×.

3.3 The bandwidth model

Once overhead is gone, decoding is bound by weight reads. Bolmo reads about 1.0 GB of weights per output byte in bf16: 0.64 GB for the global model, read once per patch, and 0.36 GB for the local stack, read on every byte. OLMo 2 reads about 0.75 GB. A fit of

$$t_{\text{byte}} = t_{\text{fixed}} + k \cdot W_{\text{byte}}, \tag{1}$$

where W_{byte} is weight bytes read per output byte, reproduces five quantized variants within about 2%, with $t_{\text{fixed}} \approx 1.15$ ms per byte. Quantization savings arrive at a steady 71 to 77% of the model's prediction; the shortfall is that fixed part, which later turned out to be per-kernel cost (Section 7).

4 Quantization: the cheapest place is the per-byte stack

Because local weights are read once per byte and global weights once per patch, quantizing the local stack saves the most time per gigabyte and costs the least quality per millisecond.

Table 2: Weight-only round-to-nearest quantization of Bolmo 1B on the graphed single-sync loop. bpb: bits per byte on WikiText-2; CUTE change in points.

Variant	ms/byte	bpb change	CUTE change
bf16	3.978	0	0
Local int8	3.422	+0.08%	+0.2
Global int8	3.175	+0.00%	−0.2
Local int4	3.132	+1.2%	−2.2
Local int4 + global int8	2.492	+1.2%	−2.4
Global int4	2.729	+3.5%	−4.2
All int4	1.992	+4.9%	−6.2

Per gigabyte of weights quantized, local layers save 4.06× more time than global layers, close to the 3.4× expected from bytes per patch (Table 2). Per millisecond saved, local int4 costs 1.4% bits per byte and global int4 2.8%. The int8 tier is essentially lossless, as for token models; local int4 with plain rounding costs about what calibrated int4 methods cost a 7B token model on WikiText-2 (+1.5 to 1.6% in cross-entropy from GPTQ and AWQ results reported by Shao et al., 2024). Calibrated quantization of the full model was not tested.

A fair comparison needs the same kernels, loop and precision rule for both models. With each model at its best precision, the ratio is 1.49× (bf16), 1.40× (int8) and 1.48× (int4). Under a rule that keeps output heads in bf16, the int4 ratio drops to 1.12×, because OLMo's 421 MB head stays expensive while Bolmo's 520-entry head is tiny: a real structural advantage of byte models, but not an even contest.

5 Why speculative drafting favours coarse units

Naive drafting makes the byte model faster but the token model much faster, because a token model verifies drafts almost for free and a byte model's rounds end and restart at byte granularity.

5.1 Drafting within patches

In speculative decoding (Leviathan et al., 2023; Chen et al., 2023) a cheap drafter proposes K units and the target model verifies them in one pass; greedy verification keeps the output exact. Our drafter is a lookup drafter: it copies a continuation from earlier context (prompt-lookup style; Saxena, 2023), otherwise uses a byte n-gram table built from 19.4 MB of text with no 32-byte overlap with any prompt. For Bolmo, a round verifies the drafted bytes with the local encoder and decoder in one CUDA graph and stops at the first patch boundary, because the global model must run before the next patch. It emits 2.0 to 2.3 bytes per round and makes Bolmo 1.19× faster in bf16 (3.995 to 3.361 ms/byte), with fp32 output identical in all 240 runs. The same drafter family on token ids makes OLMo 2 1.89× faster (2.688 to 1.420), so the ratio grows from 1.49× to 2.37×.

5.2 The verification asymmetry

At batch 1, a forward pass reads every weight once whether it processes one position or nine, and the GPU's compute is mostly idle; OLMo verifies 9 tokens for 8% more than 1. Bolmo can batch only its local stack, about 37 to 40% of its time, while the global model still runs once per patch. Its mLSTM recurrence also steps through drafted positions in sequence, and small matrix multiplications with 2 to 9 rows hit a gap in standard kernels: cuBLAS switches from gemv to padded 64×64-tile gemm, so one gate projection goes from 3.5 to 10.8 μs. A fused multi-step recurrence kernel cut the recurrence from 16.1 to 4.5 μs per extra position, but gained only about 1% end to end.

5.3 Free unit and restart position

The same small-model drafter continuations give OLMo 2 about 2.7× the bytes per round that they give Bolmo. A protocol audit and a two-by-two comparison (each model's text under each model's acceptance rule) locate the cause in the round protocol, not the text (Table 3).

Table 3: Decomposition of the acceptance gap, SmolLM2-360M drafter.

Factor	OLMo vs Bolmo, bytes per round
Text predictability	0.89×
Boundary flags must match (Bolmo only)	1.05×
Free unit per round and restart position	2.6×
Byte vs token matching	1.1×

Every round ends with one unit the target predicts itself: a token of about 4 bytes for OLMo, one byte for Bolmo. And 68% of Bolmo's rounds then restart inside a token, where a token-level drafter's first guess is right 17 to 23% of the time, against 77 to 86% elsewhere. Granularity explains only part of that: rounds restart where the drafter has just missed, and misses cluster at hard positions, which in a byte model fall mid-token. This is the tokenization bias described by Phan et al. (2025) appearing inside speculative decoding. By the chain rule a byte model does not make more hard guesses than a token model; it makes more steps, and its drafting rounds are cut at byte granularity.

6 Patch-aligned drafting

Gating cross-patch drafts to rounds that start at a patch boundary makes Bolmo 2.06× faster than its plain loop in bf16 and brings the drafted byte tax below its undrafted level.

6.1 Design

Drafts that cross patch boundaries would batch the global model too: one global pass over eight patches costs only 1.08× a pass over one (7.69 to 8.33 ms in bf16). Crossing in every round gains little, though, because each round then pays a speculative global pass that is lost when the draft is rejected. Small token-model drafters (SmolLM2-135M and -360M, Qwen2.5-0.5B) do not fix this: they emit at most 4.5 bytes per round and cost 3.7 to 5.5 ms per drafted token, more than a whole Bolmo round. Bolmo's boundary flags, however, follow the OLMo 2 tokenizer's segmentation of its own output on 98.7% of byte positions, so a lookup draft can be given flags by re-tokenizing it.

Boundary-gated drafting uses two kinds of round. A round that starts mid-patch drafts only to the end of the current patch with the lookup drafter, with no global pass. A round that starts at a boundary drafts across patches, takes flags from re-tokenization, and runs one global pass over all speculative patches. Acceptance requires both byte and flag to match; rollback restores the recurrent states and the global cache position.

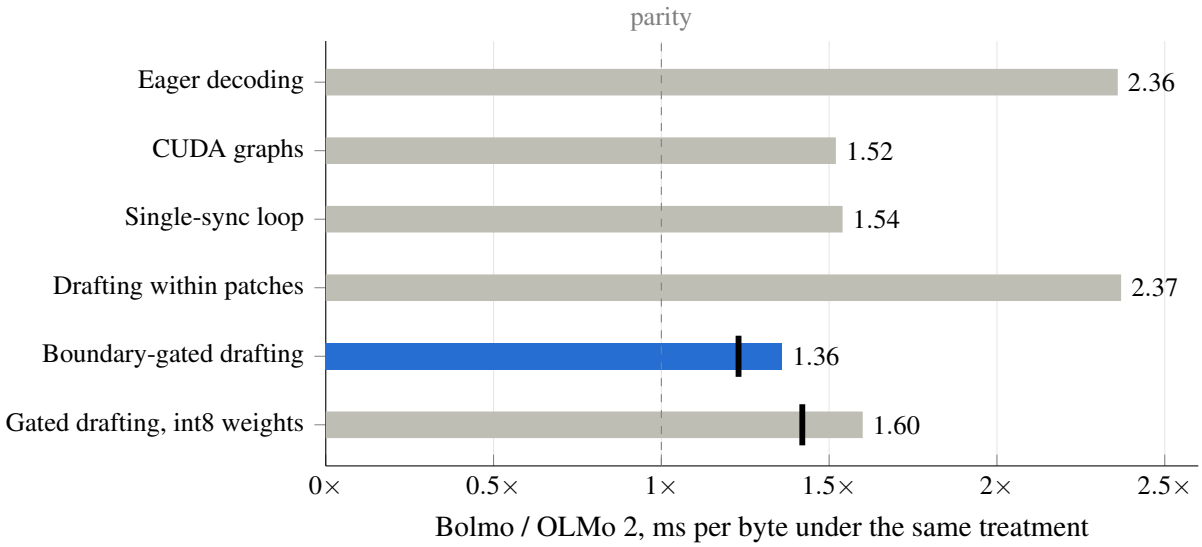


Figure 1: The byte tax across optimization stages. Generic steps leave the ratio near 1.5× or widen it; boundary-gated drafting is the only step that moves it toward parity. Black ticks mark the ratio if both models ran at their weight-bandwidth floor (Section 7). Last row: int8 global weights for Bolmo, int8 blocks for OLMo 2, skinny kernels for both. bf16 unless noted.

6.2 Results

Table 4: Boundary-gated drafting, default prompts, ms per generated byte.

Precision	Plain loop	Within-patch	Boundary-gated	Gain over within-patch
bf16	3.975	2.798	1.934	+30.9%
all-int4	1.998	1.686	1.456	+13.6%

Bytes per global pass rise from about 3.8 to 7.2, matching the simulation exactly on the tuning prompts (Table 4). A crossing round costs at most 0.10 ms beyond the pieces the cost model already contained, and the model predicted the bf16 and int4 results within 12.6% and 0.1%. In bf16, near-tie divergences change the text of some prompts, and drafted text tends to be more draftable; on prompts whose output equals the plain loop's, the gain over within-patch drafting is 37.9% (bf16) and 15.1% (int4). In fp32 the output is identical in bytes and flags for all 30 prompts at 256 and 1,024 bytes.

In the same session, drafted Bolmo is 1.36× drafted OLMo 2 (1.930 against 1.419 ms/byte), against 1.48× with neither drafting. Aligning rounds to the byte model's own structure is the one optimization in this study that favours the byte model (Figure 1).

7 How close to the floor

Weight-read optimizations favour the token model, and at int8 the byte tax sits about 13% above the bandwidth floor of this card; the overhead left is kernel count, which a persistent megakernel would barely reduce.

7.1 Amdahl's law at work

Skinny matrix kernels that read each weight once for 1 to 64 rows make int8 global weights usable inside drafting rounds: int8 reaches 77 to 91% of copy bandwidth at one row, and eight rows cost 0.97 to 1.05× one row. Bolmo with int8 global and bf16 local weights reaches 1.588 ms/byte at +0.002% bits per byte, 17.9% faster than bf16 boundary-gated drafting (25.0% on identical text). OLMo 2 gains 26 to 30% from the same kernels, because almost all of its time is weight streaming, and the ratio widens to 1.60× (Table 5).

Table 5: Which model each optimization favours.

Optimization	Gains more	Why
Drafting within patches	OLMo 2	verification amortizes its whole model
int8 weights, skinny kernels	OLMo 2	its time is almost all weight reads
Boundary-gated drafting	Bolmo	removes a cost only byte models have

7.2 Kernel census and the megakernel

A bf16 local round takes 1,385 μs against an 877 μs weight-read floor. Its 42 matmul kernels run within 4% of the floor; the excess is 238 small non-matmul kernels (371 μs: norms, gate arithmetic, residuals, recurrence bookkeeping) and 74 μs of gaps. Persistent megakernels can remove kernel boundaries (Hazy Research, 2025; Cheng et al., 2026), so we measured their mechanisms on this card. A counter barrier between phases costs about 1 μs; dataflow waits save only 3 to 9% of that; prefetching weights changes achieved bandwidth by -1 to +2 points; and a persistent kernel ran the same 28 fused phases in 1,123 μs against 1,126 as a CUDA graph. The per-phase stall is the arrival of the slowest block's result, which prefetching does not hide. A real local megakernel was predicted to gain 4 to 8% and was not built.

7.3 The floor

At the bandwidth floor, time per byte is weight bytes read per output byte. With drafting, Bolmo reads its global weights about 0.15 times per byte and its local weights about 0.33 times; OLMo reads its blocks and head about 0.13 times. From the parameter counts and these rates we obtain the floor ratios in Table 6.

Table 6: Bolmo / OLMo 2 ratio if both models ran at their weight-bandwidth floor, with drafting.

Precision	Local at same precision	Local in int4
bf16	1.23×	0.97×
int8	1.42×	1.24×

The measured ratios sit 11% (bf16, 1.36×) and 13% (int8, 1.60×) above their floors. The last candidates, a quantized local stack with int8 global weights, were predicted at +6.1% and +7.0% and stopped at an 8% gate.

8 Discussion and limitations

The byte tax is best understood as a property of a model and its optimization stack together: generic optimizations favour token models, and only structure-aware ones close the gap.

Three observations generalize beyond Bolmo. First, at batch 1, the per-byte local stack is both the source of the byte tax and the cheapest part of a byte model to quantize. Second, speculative decoding carries a granularity bias: the free unit per round and the restart position favour whatever model has the coarser output unit, independent of model quality. Third, the split into a small, frequently read local stack and a large, rarely read global model is a structural asset on hardware with large on-chip caches, where a quantized local stack (about 90 MB in int4) could come close to staying resident; on this card's 4 MB L2 it cannot.

Limitations.

- One GPU, one operating system, batch size 1, greedy decoding, models at the 1B scale. The T4 cross-check reproduces the eager ratio but not absolute numbers. Batching would hide much of the launch overhead; sampled decoding was not tested.
- Drafting acceptance is flattered by repetitive greedy output from 1B models (half of all 8-byte windows repeat within 256 bytes).
- In bf16 and int4, near-tie divergences change the generated text between variants, which mixes a speed effect and a text effect. Identical-output subsets are reported but small.
- Bolmo's bits per byte include coding the boundary flags, so it compares across Bolmo variants only.
- Quantization used round-to-nearest; calibrated methods may change the quality numbers. CUTE intervals are about ± 2 points.
- Some comparisons combine cells from different sessions; same-session blocks were run for every headline ratio.
- The floor ratios rest on parameter counts and measured rates, not on runs at the floor.
- The extended English, German and Unicode prompts were written for the study; Python and JSON prompts are real files.

9 Conclusion

On a consumer GPU at batch size 1, a byteified 1B model can be made 10.3× faster than its eager baseline and brought to within 1.36 to 1.60× of its token-level source, 11 to 13% above what weight bandwidth allows. Most of the eager gap is overhead in the per-byte local loop, which CUDA graphs and a single-sync loop remove. The rest follows weight reads, and the per-byte stack is the cheapest part to shrink. Speculative drafting, applied naively, widens the gap because it favours coarse units; aligned to patch boundaries, it narrows the gap below its undrafted level. Byte-level models need decoding methods built around their own structure, not methods inherited from token models.

Acknowledgements

The experiments were implemented and run by Claude Code (Anthropic) under written task specifications; study design, hypotheses and gates were fixed in those specifications before each measurement. Interpretation and any errors are the author's.

Code and data availability

The profiling harness, kernels, configurations, task specifications, raw results and the full findings log will be released with the Zenodo record of this preprint. The companion technical report lists all 67 hypotheses with verdicts and documents every study, including the negative results.

References

Maximilian Beck, Korbinian Pöppel, Phillip Lippe, Richard Kurle, Patrick M. Blies, Günter Klambauer, Sebastian Böck, and Sepp Hochreiter. xLSTM 7B: A recurrent LLM for fast and efficient inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. arXiv:2302.01318.

Xinhao Cheng, Zhihao Zhang, Yu Zhou, Jianan Ji, Jinchen Jiang, Zepeng Zhao, Ziruo Xiao, Zihao Ye, Yingyi Huang, Ruihang Lai, Hongyi Jin, Bohan Hou, Mengdi Wu, Yixin Dong, Anthony Yip, Zihao Ye, Songting Wang, Wenqin Yang, Xupeng Miao, Tianqi Chen, and Zhihao Jia. MPK: A compiler and runtime for mega-kernelizing tensor programs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026. arXiv:2512.22219.

Lukas Edman, Helmut Schmid, and Alexander Fraser. CUTE: Measuring LLMs' understanding of their tokens. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

Hazy Research. Look ma, no bubbles! designing a low-latency megakernel for Llama-1B. `https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles`, 2025. Blog post, May 2025.

Sukjun Hwang, Brandon Wang, and Albert Gu. Dynamic chunking for end-to-end hierarchical sequence modeling. In *International Conference on Learning Representations (ICLR)*, 2026.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

Kalyani Marathe, Artidoro Pagnoni, Tomasz Limisiewicz, Margaret Li, Mike Lewis, Luke Zettlemoyer, and Srinivasan Iyer. Breaking the token ceiling: Distilling smaller, stronger byte models, 2026. arXiv:2609.12303.

Benjamin Minixhofer, Tyler Murray, Tomasz Limisiewicz, Anna Korhonen, Luke Zettlemoyer, Noah A. Smith, Edoardo M. Ponti, Luca Soldaini, and Valentin Hofmann. Retrofitting language models to operate over bytes. *Nature*, 658:915–922, 2026. doi: 10.1038/s41586-026-11111-4.

Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2412.09871.

Buu Phan, Brandon Amos, Itai Gat, Marton Havasi, Matthew Muckley, and Karen Ullrich. Exact byte-level probabilities from tokenized language models for FIM-tasks and model ensembles. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.09303.

Apoorv Saxena. Prompt lookup decoding. GitHub repository, 2023.

Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.13137.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Allyson Ettinger, Michal Guerquin, David Heineman, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Jake Poznanski, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 OLMo 2 furious. In *Second Conference on Language Modeling (COLM)*, 2025. arXiv:2501.00656.

Jie Wang, Shiwei Luo, Qi Zhang, and Yuanbin Wu. Byte language models: Scaling, emergent abstractions, and information allocation, 2026. arXiv:2610.05978.